

bok25

基  
礎  
班

、**考研数据結構**



# 归并排序





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程  
将两个有序表合并成一个有序表的过程称为2路归并



# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

## (二路归并排序) 基本思想

假设初始序列含有 $n$ 个记录，则可将其看成 $n$ 个有序的子序列，每个子序列的长度为1，然后两两归并，得到 $\lceil n/2 \rceil$ 个长度为2或1的有序子序列；再两两归并，如此重复，直至得到一个长度为 $n$ 的有序序列为止





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 49 | 38 | 65 | 97 | 76 | 13 | 27 |
|----|----|----|----|----|----|----|



# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

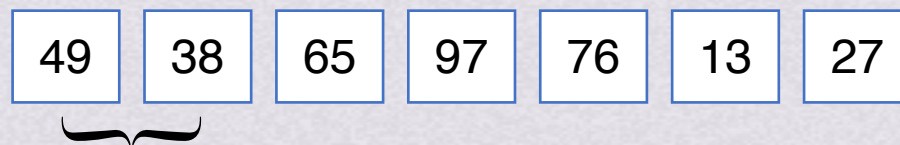
|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 49 | 38 | 65 | 97 | 76 | 13 | 27 |
|----|----|----|----|----|----|----|





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程







# 归并排序

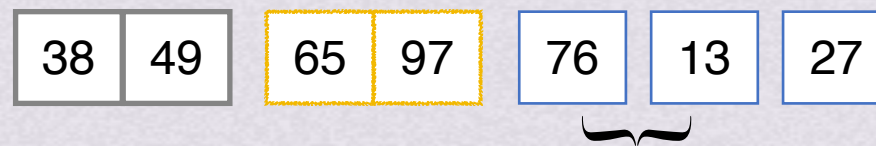
归并排序：将两个或两个以上的有序表合并成一个有序表的过程





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程







# 归并排序

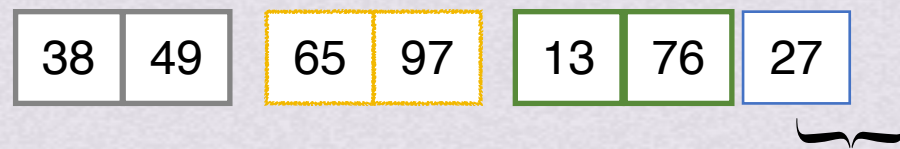
归并排序：将两个或两个以上的有序表合并成一个有序表的过程





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

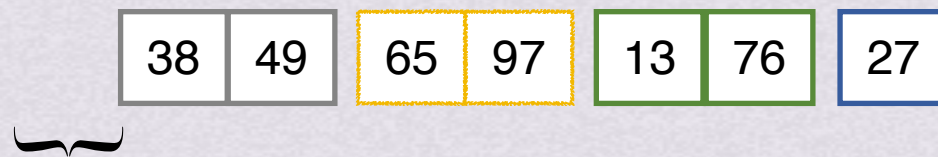






# 归并排序

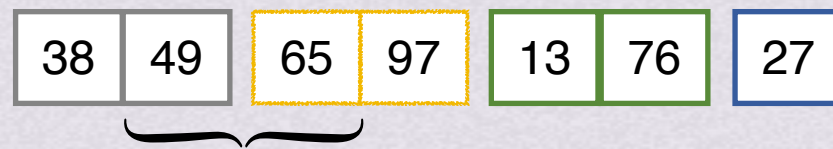
归并排序：将两个或两个以上的有序表合并成一个有序表的过程





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

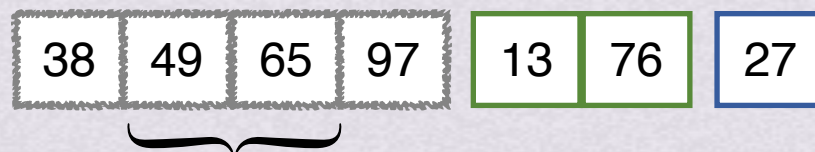






# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程







# 归并排序

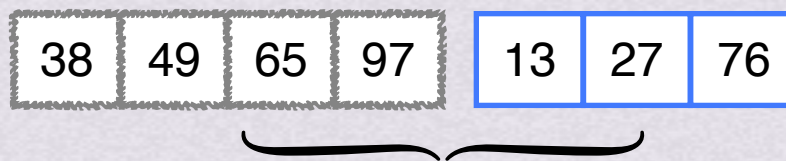
归并排序：将两个或两个以上的有序表合并成一个有序表的过程





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

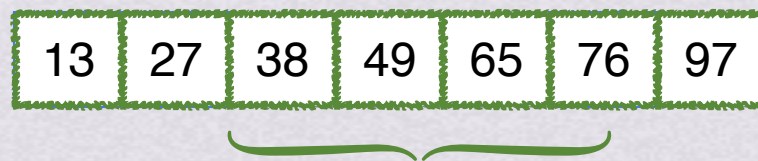






# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程



经过三趟归并，排序完成



# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

```
void MergeSort(SqList &L) { // 对顺序表 L 进行归并排序
```

```
MSort(L, L, 1, L.length); } // 对L进行归并排序，排序结果放回L中，low=1，high=L.length
```

```
void MSort(RedType R[], RedType T[], int low, int high)
```

```
{ // R[low..high] 归并排序后放入 T[low..high] 中
```

```
if (low == high) T[low] = R[low];
```

```
else
```

```
{
```

```
mid = (low + high) / 2; // 将当前序列一分为二，求出分裂点 mid
```

```
MSort(R, S, low, mid); // 对子序列 R[low..mid] 递归进行归并排序
```

```
// 结果放入 S[low..mid]
```

```
MSort(R, S, mid + 1, high); // 对子序列 R[mid + 1..high] 递归进行归并排序，
```

```
// 结果放入 S[mid + 1..high]
```

```
Merge(S, T, low, mid, high); // 将 S[low..mid] 和 S[mid + 1..high] 归并到 T[low..high]
```

```
}
```

```
void Merge(RedType R[], RedType T[], int low, int mid, int high)
```

```
{ // 将有序表 R[low..mid] 和 R[mid + 1..high] 归并为有序表 T[low..high]
```

```
i = low; j = mid + 1; k = low;
```

```
while (i <= mid && j <= high) // 将R中的记录由小到大并入T中
```

```
{
```

```
if (R[i] <= R[j]) T[k++] = R[i++];
```

```
else T[k++] = R[j++];
```

```
}// while
```

```
while (i <= mid) T[k++] = R[i++]; // 将剩余的R[i..mid]复制到T中
```

```
while (j <= high) T[k++] = R[j++]; // 将剩余的R[j..high]复制到T中
```

```
}
```

将表不断的平均分，递归的进行归并排序

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 13 | 27 | 38 | 49 | 65 | 76 | 97 |
|----|----|----|----|----|----|----|

对比挑选更小的元素不断的放进目标数组

两部分用于归并的数据中有一部分用完了，另一部分的剩余就可以直接复制到结果中  
两个while只有一个能够被执行





# 归并排序

归并排序：将两个或两个以上的有序表合并成一个有序表的过程

```
void MergeSort(SqList &L) { // 对顺序表 L 进行归并排序
    MSort(L, L, 1, L.length); }
```

```
void MSort(RedType R[], RedType T[], int low, int high)
{ // R[low..high] 归并排序后放入 T[low..high] 中
    if (low == high) T[low] = R[low];
    else
    {
        mid = (low + high) / 2; // 将当前序列一分为二，求出分裂点 mid
        MSort(R, S, low, mid); // 对子序列 R[low..mid] 递归进行归并排序
        // 结果放入 S[low..mid]
        MSort(R, S, mid + 1, high); // 对子序列 R[mid+1..high] 递归进行归并排序，
        // 结果放入 S[mid+1..high]
        Merge(S, T, low, mid, high); // 将 S[low..mid] 和 S[mid+1..high] 归并到 T[low..high]
    }
}
```

```
void Merge(RedType R[], RedType T[], int low, int mid, int high)
{ // 将有序表 R[low..mid] 和 R[mid+1..high] 归并为有序表 T[low..high]
    i = low; j = mid + 1; k = low;
    while (i <= mid && j <= high) // 将 R 中的记录由小到大并入 T 中
    {
        if (R[i] <= R[j]) T[k++] = R[i++];
        else T[k++] = R[j++];
    } // while
    while (i <= mid) T[k++] = R[i++]; // 将剩余的 R[i..mid] 复制到 T 中
    while (j <= high) T[k++] = R[j++]; // 将剩余的 R[j..high] 复制到 T 中
}
```

时间复杂度  $O(n \log_2 n)$

空间复杂度  $O(n)$

**稳定！**



# 基数排序





# 基数排序

## 基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的



# 基数排序

## 基本思想

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的

## 链式基数排序

假设关键字均为 $[0,999]$ 之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

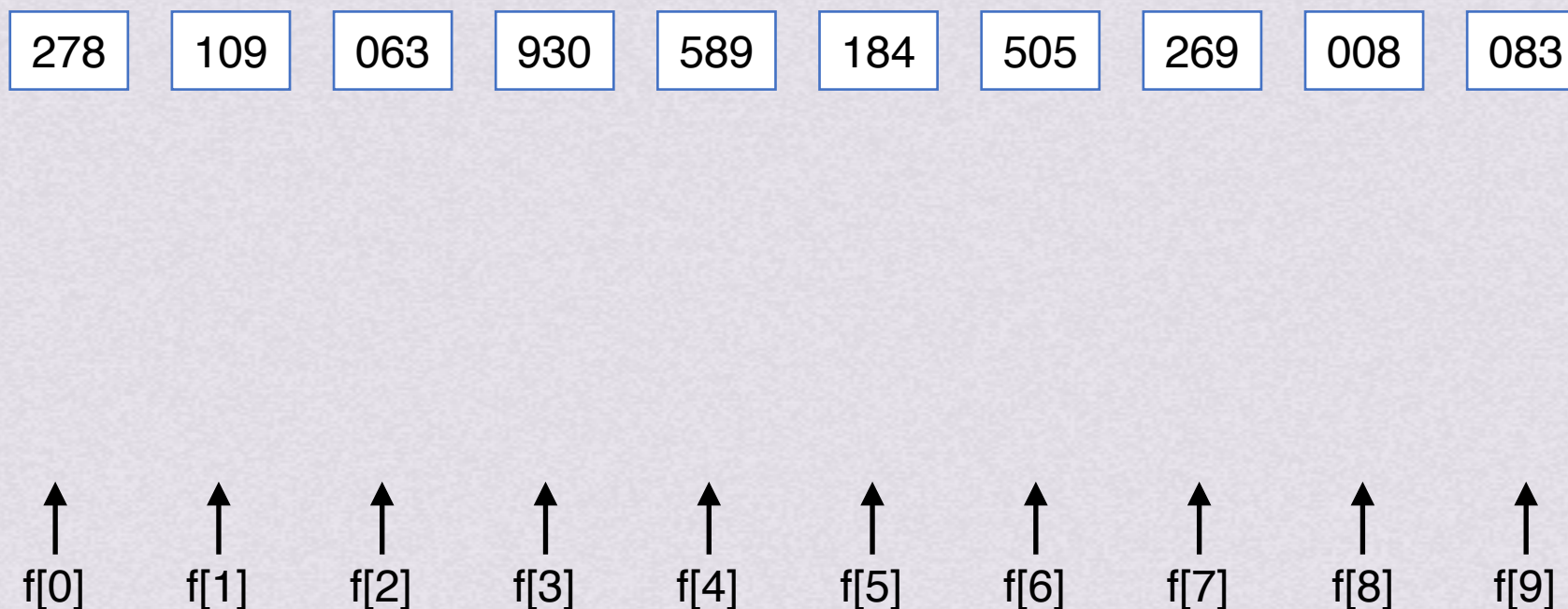




**基数排序** 不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

进行第一趟“分配”

将数据按照个位数分配到10个队列





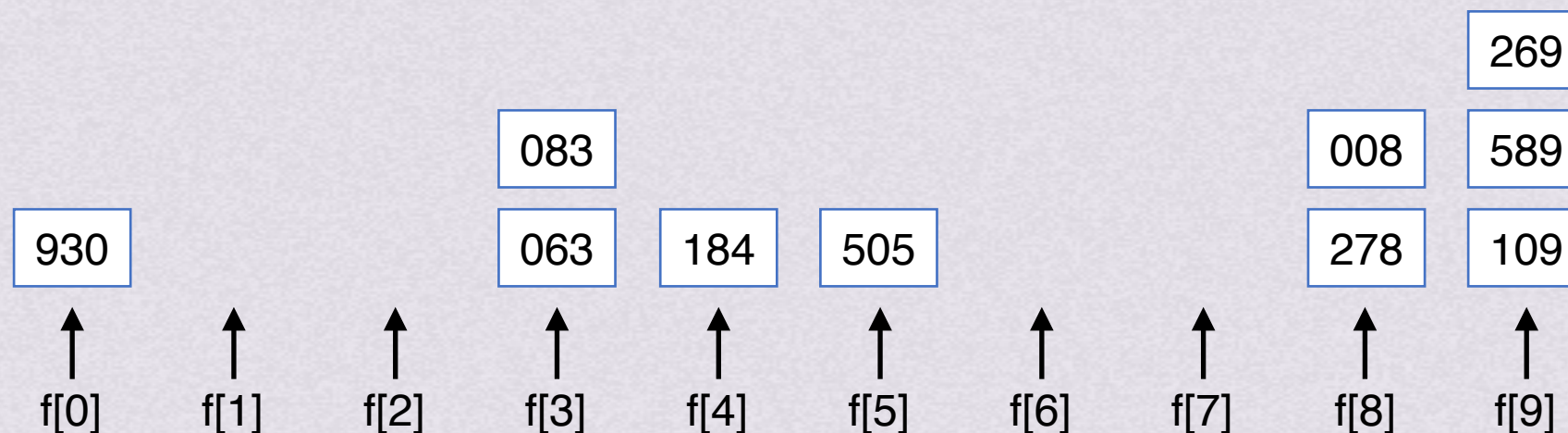
# 基数排序

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

进行第一趟“分配”

将数据按照个位数分配到10个队列

再按照队列顺序依次收集







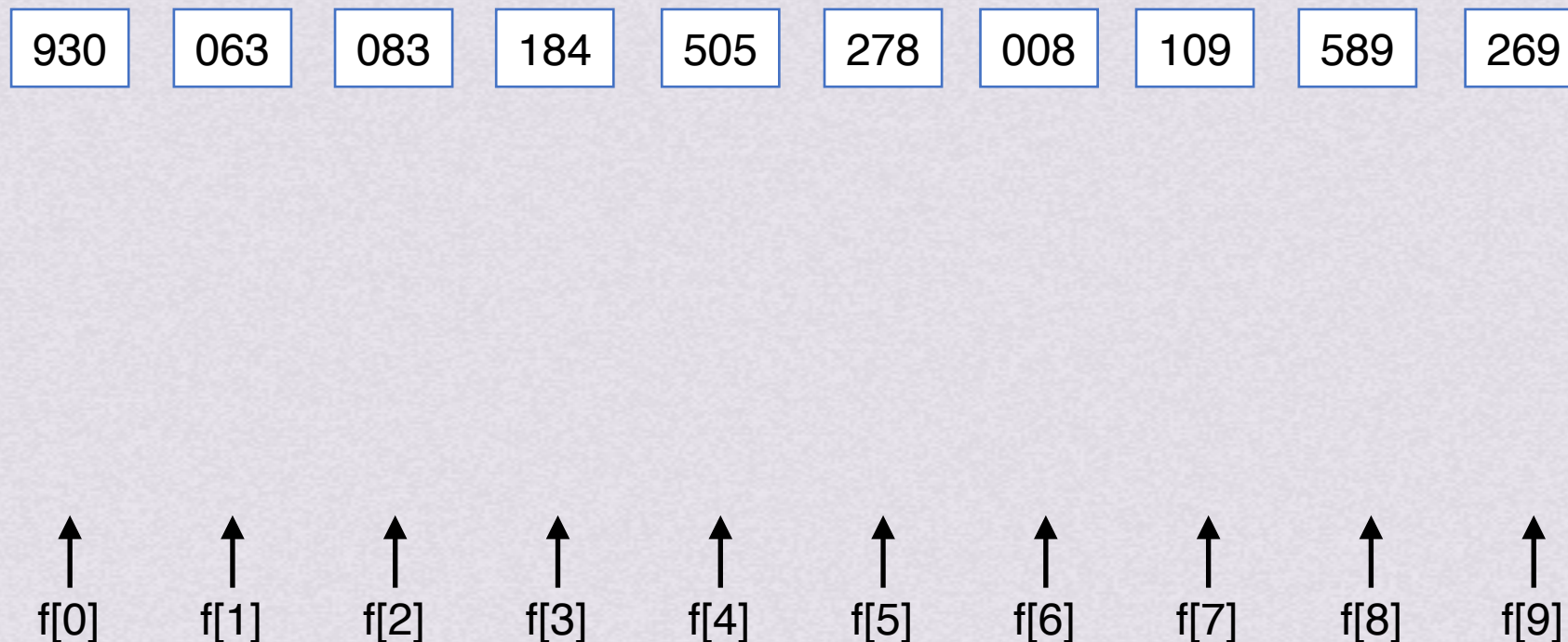
# 基数排序

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

进行第一趟“分配”

将数据按照个位数分配到10个队列

再按照队列顺序依次收集





**基数排序** 不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

进行第一趟“分配”

将数据按照个位数分配到10个队列

再按照队列顺序依次收集

再将数据按照十位数分配到10个队列

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 930 | 063 | 083 | 184 | 505 | 278 | 008 | 109 | 589 | 269 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

|      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|
| ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    |
| f[0] | f[1] | f[2] | f[3] | f[4] | f[5] | f[6] | f[7] | f[8] | f[9] |





# 基数排序

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

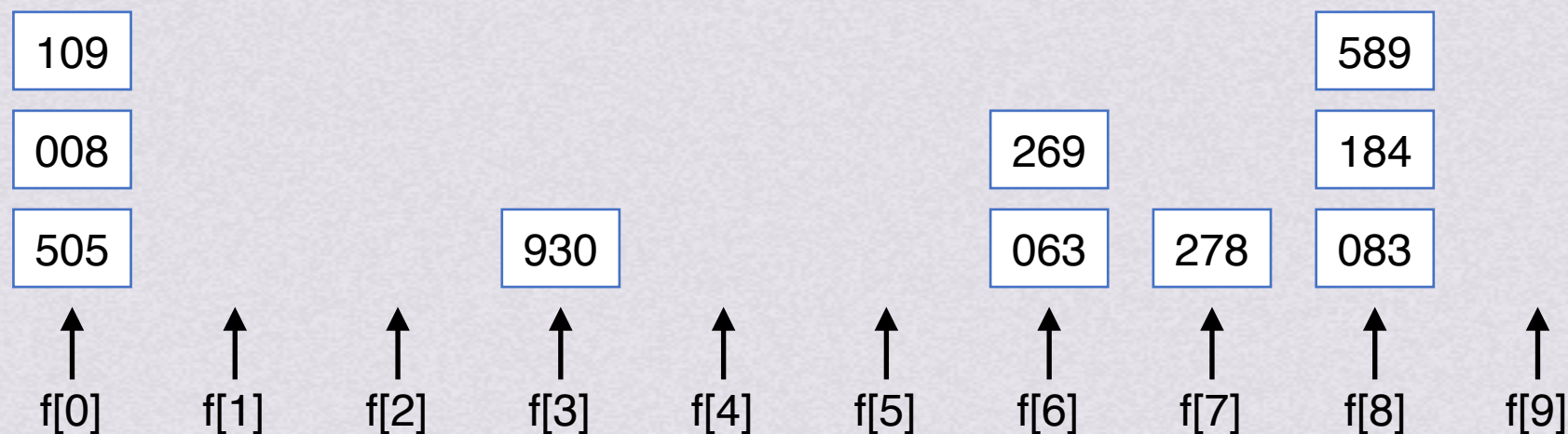
进行第一趟“分配”

将数据按照个位数分配

再按照队列顺序依次收集

再将数据按照十位数分配

再按照队列顺序依次收集





**基数排序** 不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

进行第一趟“分配”

将数据按照个位数分配到10个队列

再按照队列顺序依次收集

再将数据按照十位数分配到10个队列

再按照队列顺序依次收集

再将数据按照百位数分配到10个队列

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 505 | 008 | 109 | 930 | 063 | 269 | 278 | 083 | 184 | 589 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

|      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|
| ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    |
| f[0] | f[1] | f[2] | f[3] | f[4] | f[5] | f[6] | f[7] | f[8] | f[9] |





# 基数排序

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序

进行第一趟“分配”

将数据按照个位数分配到10个队列

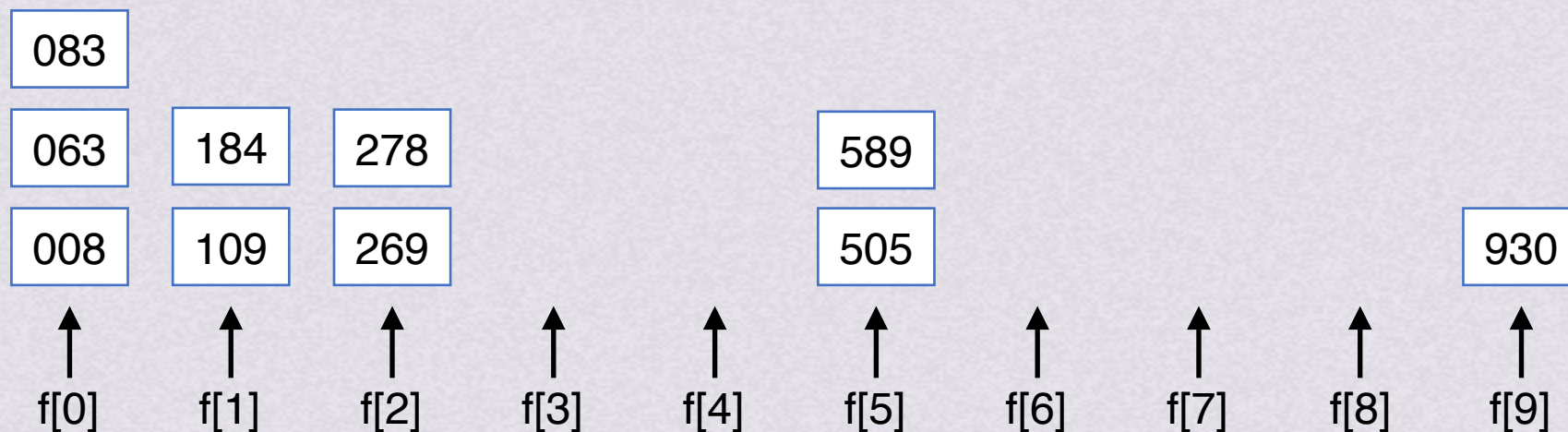
再按照队列顺序依次收集

再将数据按照十位数分配到10个队列

再按照队列顺序依次收集

再将数据按照百位数分配到10个队列

再按照队列顺序依次收集





# 基数排序

不比较关键字的大小，根据关键字中各位的值，对排序记录进行若干趟“分配”和“收集”实现排序的  
假设关键字均为[0,999]之间的关键字，先按照个位数排序，再按照十位数排序，最后按百位数排序  
进行第一趟“分配”

将数据按照个位数分配到10个队列

再按照队列顺序依次收集

再将数据按照十位数分配到10个队列

再按照队列顺序依次收集

再将数据按照百位数分配到10个队列

再按照队列顺序依次收集

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 008 | 063 | 083 | 109 | 184 | 269 | 278 | 505 | 589 | 930 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

排序完成！

|      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|
| ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    | ↑    |
| f[0] | f[1] | f[2] | f[3] | f[4] | f[5] | f[6] | f[7] | f[8] | f[9] |





# 基数排序

```
void Distribute(SLCell &r, int i, ArrType &f, ArrType &e)
{//静态链表L的r域中记录已按(keys[0], ..., keys[i-1])有序
//本算法按第i个关键字keys[i]建立RADIX个子表, 使同一子表中记录的 keys[i] 相同
//f[0..RADIX-1] 和 e[0..RADIX-1] 分别指向各子表中第一个和最后一个记录
    for(j=0; j<RADIX; ++j) f[j]=0; //各子表初始化为空表
    for(p=r[0].next; p; p=r[p].next)
    {
        j=ord(r[p].keys[i]); //ord 将记录中第 i 个关键字映射到 [0..RADIX-1]
        if(!f[j]) f[j]=p;
        else r[e[j]].next=p;
        e[j]=p; // 将 p 所指的节点插入第 j 个子表中
    } //for
}
```

```
void Collect(SLCell &r, int i, ArrType f, ArrType e)
{// 本算法按 keys[i] 自小至大地将 f[0..RADIX-1] 所指各子表依次链接成一个链表
//e[0..RADIX-1] 为各子表的尾指针
    for(j=0; !f[j]; j=succ(j)); //找第一个非空子表, succ()为求后继函数
    r[0].next=f[j]; t=e[j]; //r[0].next指向第一个非空子表中第一个节点
    while(j<RADIX)
    {
        for(j=succ(j); j<RADIX-1&&!f[j]; j=succ(j)); //找下一个非空子表
        if(f[j]) {r[t].next=f[j]; t=e[j];} //链接两个非空子表
    } //while
    r[t].next=0; //t指向最后一个非空子表中的最后一个节点
}
```

## 分配代码

```
void RadixSort(SLList &L) { //L 是采用静态链表表示的顺序表
//对L进行基数排序, 使得L成为按关键字自小到大的有序静态链表, L.r[0]为头节点
    for(i=0; i<L.recnum; ++i) L.r[i].next=i+1;
    L.r[L.recnum].next = 0; //将L改造为静态链表
    for(i=0; i<L.keynum; ++i)
    { //按最低位优先依次对各关键字进行分配和收集
        Distribute(L.r, i, f, e); // 第 i 趟分配
        Collect(L.r, i, f, e); // 第 i 趟收集
    } //for
}
```

## 基数排序代码

## 收集代码



# 基数排序

```
void Distribute(SLCell &r, int i, ArrType &f, ArrType &e)
{//静态链表L的r域中记录已按(keys[0], ..., keys[i-1])有序
//本算法按第i个关键字keys[i]建立RADIX个子表, 使同一子表中记录的 keys[i] 相同
//f[0..RADIX-1] 和 e[0..RADIX-1] 分别指向各子表中第一个和最后一个记录
for(j=0; j<RADIX; ++j) f[j]=0; //各子表初始化为空表
for(p=r[0].next; p; p=r[p].next)
{
    j=ord(r[p].keys[i]); //ord 将记录中第 i 个关键字映射到 [0..RADIX-1]
    if(!f[j]) f[j]=p;
    else r[e[j]].next=p;
    e[j]=p; // 将 p 所指的节点插入第 j 个子表中
} //for
}
```

```
void Collect(SLCell &r, int i, ArrType f, ArrType e)
{// 本算法按 keys[i] 自小至大地将 f[0..RADIX-1] 所指各子表依次链接成一个链表
//e[0..RADIX-1] 为各子表的尾指针
for(j=0; !f[j]; j=succ(j)); //找第一个非空子表, succ()为求后继函数
r[0].next=f[j]; t=e[j]; //r[0].next指向第一个非空子表中第一个节点
while(j<RADIX)
{
    for(j=succ(j); j<RADIX-1&&!f[j]; j=succ(j)); //找下一个非空子表
    if(f[j]) {r[t].next=f[j]; t=e[j];} //链接两个非空子表
} //while
r[t].next=0; //t指向最后一个非空子表中的最后一个节点
}
```

空间复杂度: 需使用 $r$ 个队列,  $O(r)$

时间复杂度: 共需 $d$ 趟分配、收集

每趟分配需要遍历所有关键字,  $O(n)$ ,

每趟收集复杂度为 $O(r)$

总复杂度为 $O(d(n + r))$

## 稳定!

```
void RadixSort(SLList &L) { //L 是采用静态链表表示的顺序表
//对L进行基数排序, 使得L成为按关键字自小至大的有序静态链表, L.r[0]为头节点
for(i=0; i<L.recnum; ++i) L.r[i].next=i+1;
L.r[L.recnum].next = 0; //将L改造为静态链表
for(i=0; i<L.keynum; ++i)
{//按最低位优先依次对各关键字进行分配和收集
    Distribute(L.r, i, f, e); // 第 i 趟分配
    Collect(L.r, i, f, e); // 第 i 趟收集
} //for
}
```